

UNIT-2

LEETCODE EXERCISE

1.Find Users with Valid E-Mails

Problem Summary:

You are given a table named Users.

Column Name	Type
user_id	int
Name	Varchar
Mail	Varchar(50)

- user_id is the primary key.
- Each row contains the ID and name (which is an e-mail address) of a user.

You need to find users whose e-mail addresses are valid

Valid EmailRules

A valid e-mail:

1. Starts with a letter (a-z or A-Z)

2. Followed by:

o letters (a-z, A-

Z) o digits (0-9)

o underscore (_)

o period (.) o

hyphen (-)

3. Ends [with:@leetcode.com](https://leetcode.com)

Task

Write an SQL query to find all users having valid e-mail addresses.

Return the result table in any order.

MySQL Solution:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT user_id, name, mail
3 FROM Users
4 WHERE REGEXP_LIKE(
5     mail,
6     '^[A-Za-z][A-Za-z0-9._-]*@leetcode\\.com$',
7     'c'
8 );
```

Output

user_id	name	mail
1	Winston	winston@leetcode.com
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com

Expected

user_id	name	mail
1	Winston	winston@leetcode.com
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com

2. Patients With a Condition

Problem Summary:

You are given a table named Patients.

Column Name	Type
patient_id	int
patient_Name	Varchar
conditions	Varchar

- patient_id is the primary key.

Important Rule

A patient is considered valid if:

- DIAB1 appears at the beginning of the string, or
- DIAB1 appears after a space.

Task

Write an SQL query to report the patient ID, patient name, and conditions of patients who have Type I Diabetes (DIAB1).

Return the result table in any order.

MySQL Solution:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     patient_id,
4     patient_name,
5     conditions
6 FROM Patients
7 WHERE conditions LIKE 'DIAB1%'
8    OR conditions LIKE '% DIAB1%';
```

Output

patient_id	patient_name	conditions
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100

Expected

patient_id	patient_name	conditions
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100

3.Second Highest Salary

Problem Summary:

You are given an Employee table.

Column Name	Type
id	int
salary	int

- id is the primary key.
- Each row contains the salary of an employee.
- Salaries may contain duplicates.

Your task is to find the second highest distinct salary.

If there is no second highest salary, return NULL.

Task :

Write an SQL query to report the second highest distinct salary from the Employee table.

SQL Solution (Using LIMIT and OFFSET):

```
MySQL ▾ 🔒 Auto
1 # Write your MySQL query statement below
2 SELECT
3 (
4     SELECT DISTINCT salary
5     FROM Employee
6     ORDER BY salary DESC
7     LIMIT 1 OFFSET 1
8 ) AS SecondHighestSalary;
```

Output

```
| SecondHighestSalary |
| ----- |
| 200 |
```

Expected

```
| SecondHighestSalary |
| ----- |
| 200 |
```

4.Customer Placing the Largest Number of Orders

Problem Summary:

You are given an Orders table.

Column Name	Type
order_number	int
customer_number	int

- order_number is unique.
- Each row represents one order placed by a customer.
- A customer can place multiple orders.

- Your task is to find the customer who placed the largest number of orders.

The problem guarantees that:

There is exactly one customer who has placed the maximum number of orders.

Task :

Write an SQL query to find the customer_number of the customer who has placed the largest number of orders.

Return the result table in any order.

MySQL Solution (GROUP BY + ORDER BY + LIMIT)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(*) DESC
6 LIMIT 1;
```

Output

customer_number

3

Expected

customer_number

3

5.Product Sales Analysis III

Problem Summary:

You are given two tables, but only the Sales table is needed for this problem. Sales Table

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

- (sale_id, year) is the primary key.
- Each row represents a sale of a product in a particular year.

Task :

Write an SQL query to report:

- product_id
- first_year
- quantity
- price

for the first year in which each product was sold.

Return the result table in any order.

MySQL Solution (GROUP BY + JOIN):

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     product_id,
4     year first_year,
5     quantity,
6     price
7 FROM Sales
8 WHERE (product_id, year) IN (
9     SELECT product_id, MIN(year)
10    FROM Sales
11   GROUP BY product_id
12 );
```

Output

product_id	first_year	quantity	price
100	2008	10	5000
200	2011	15	9000

Expected

product_id	first_year	quantity	price
100	2008	10	5000
200	2011	15	9000